

1.1. Introduction:

In this study, we address the problem of **automated meeting transcript generation and documentation**, where raw meeting audio must be converted into an accurate and usable record of the discussion. The primary objective is to reliably recover the content of the meeting despite transcription errors, conversational speech, multiple speakers, and technical terminology. Following this, the system refines the transcript and generates structured documentation which contains the key discussion points, decisions, and action items, while preserving the factual information present in the original meeting and avoiding unsupported assumptions.

1.2. Literature Review:

1.2.1. Rennard et al. [1]

Rennard et al. [1] present a comprehensive survey of **abstractive meeting summarization**, highlighting challenges such as noisy and disfluent speech, multiple speakers, long conversations, and maintaining factual consistency. They organize existing approaches using an **Interpretation–Transformation–Generation** framework, in which the conversation is first interpreted, then transformed into a structured representation, and finally converted into a summary. The survey emphasizes the importance of speaker information, topic segmentation, and conversational context for producing reliable meeting summaries, particularly when identifying decisions and other important information. These observations motivate our use of an intermediate **Transformation layer**, which organizes the refined transcript into evidence-linked topics, discussion units, decisions, and action items before generating the final meeting documentation.

1.2.2. Du et al. [2]

Du et al. [2] investigate **speaker diarization for multi-party meetings**, with particular emphasis on the challenging case of **overlapping speech**, where multiple speakers talk simultaneously. They propose **Speaker Overlap-aware Neural Diarization (SOND)**, which explicitly models overlapping speaker activity and speaker dependencies to improve speaker attribution. Our system addresses the same speaker-attribution challenge using **PyAnnote Community-1** for speaker diarization, followed by alignment with timestamped ASR output.

1.2.3. Jin et al. [3]

Jin et al. [3] propose **CFAS (Consensus-Focused Abstractive Summarization)**, a framework that explicitly considers **topics and consensus points** when generating meeting summaries. Their approach incorporates topic segmentation and consensus identification before performing consensus-aware summarization, giving greater importance to utterances that represent agreements or decisions. Inspired by this work, we extended our Transformation stage with **consensus analysis**, allowing extracted decisions to be evaluated against supporting conversational evidence. The system distinguishes confirmed agreements from proposals, contested discussions, and unresolved decisions rather than treating every decision-like statement as a final decision.

1.2.4. Inoue et al. [4]

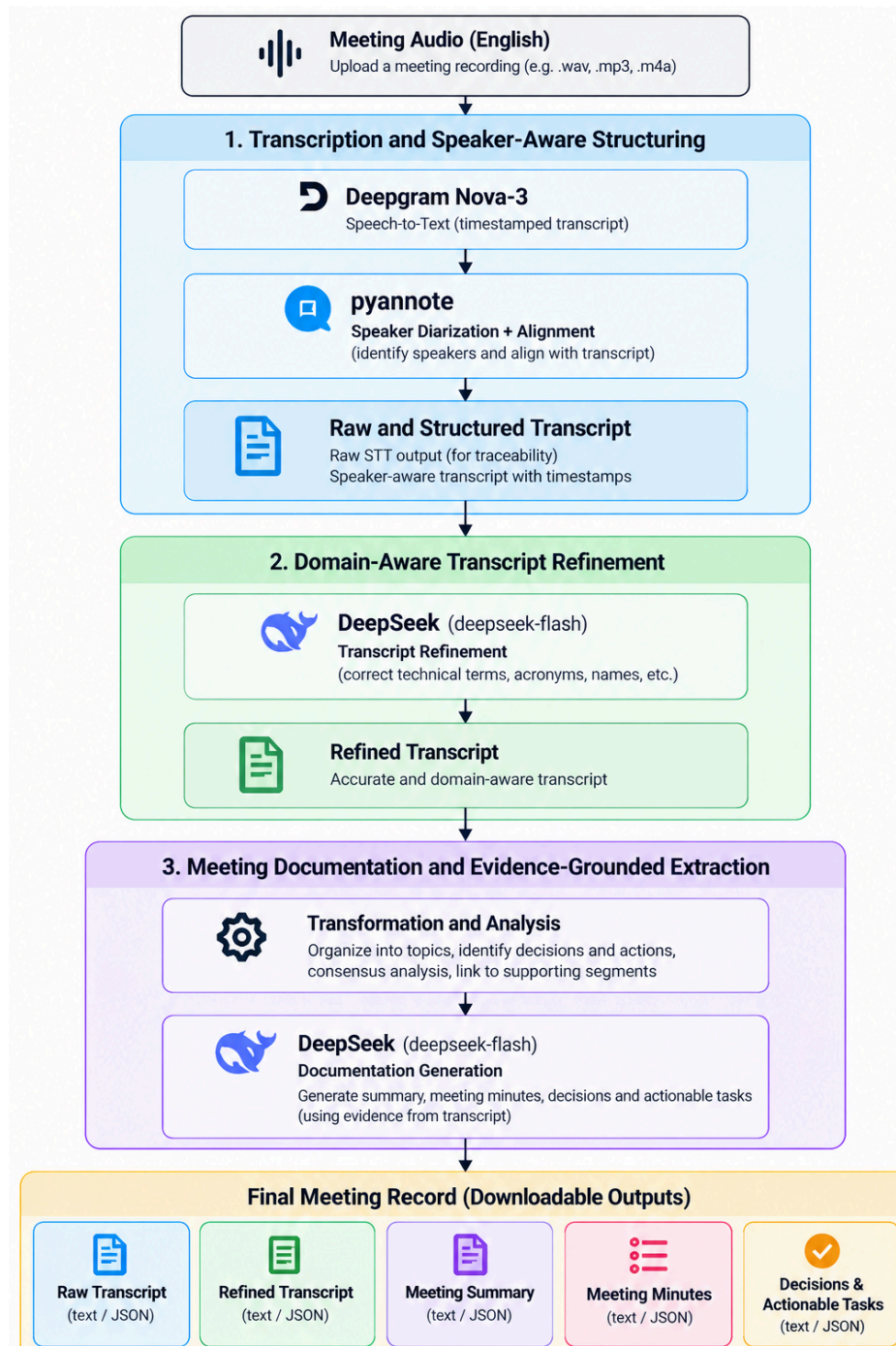
Inoue et al. [4] introduce the **Meeting Decision Tracker**, which identifies decision-related utterances and rewrites them into **self-contained decision statements**. Their work addresses the problem that spoken decisions are often context-dependent, with important information omitted or represented through references that require surrounding dialogue for interpretation. By incorporating conversational context, their system reconstructs these utterances into clearer written decisions. Inspired by this approach, we added contextual reconstruction to our Transformation stage, using surrounding transcript segments to make extracted decisions self-contained while retaining links to the original evidence.

REFERENCES:

- [1] V. Rennard, G. Shang, J. Hunter, and M. Vazirgiannis, "Abstractive Meeting Summarization: A Survey," *Transactions of the Association for Computational Linguistics*, vol. 11, 2023. [TransACL](#)
- [2] Z. Du, S. Zhang, S. Zheng, and Z.-J. Yan, "Speaker Overlap-aware Neural Diarization for Multi-party Meeting Analysis," *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 7458–7469, 2022. [ACL Anthology](#)
- [3] Y. Jin, Q. Shi, and Q. Liu, "CFAS: Consensus-Focused Abstractive Meeting Summarization through Multi-Party Discourse Modeling," *Journal of King Saud University Computer and Information Sciences*, vol. 37, art. 187, 2025. [Springer](#)
- [4] S. Inoue, H. Nguyen, H. Pham, T. Liu, and M.-T. Nguyen, "Meeting Decision Tracker: Making Meeting Minutes with De-Contextualized Utterances," *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing: System Demonstrations*, pp. 88–94, 2022.

1.3 Methodology:

The proposed system processes a meeting recording through three main stages: **transcription and speaker diarization, transcript refinement, and meeting documentation**. The stages are executed sequentially so that each stage works on the output of the previous one.



1.3.1 Transcription and Speaker-Aware Structuring

The first stage converts the English meeting audio into a timestamped transcript using **Deepgram Nova-3**. The raw speech-to-text output is retained as a separate artifact and is not overwritten by later processing.

To preserve conversational structure, **PyAnnote Community-1** is used for speaker diarization. The diarization output is aligned with the timestamped transcript so that each transcript segment retains its associated speaker and temporal information. This produces a structured transcript containing fields such as speaker, start time, end time, and segment text. The metadata is retained in the machine-readable representation even though the human-readable transcript can omit speaker labels and timestamps.

This stage therefore provides two representations of the same meeting: the original raw transcript for traceability and a structured speaker-aware transcript for downstream processing.

1.3.2 Domain-Aware Transcript Refinement

The second stage uses a **DeepSeek *deepseek-flash*** language model to refine the structured transcript. Its role is limited to correcting plausible speech-recognition errors, particularly technical terms, acronyms, names, identifiers, and domain-specific terminology. It is not used to summarize or reinterpret the discussion.

The refinement process is deliberately conservative. Speaker information, ordering, timestamps, numbers, negation, uncertainty, questions, proposals, and commitments must be preserved. The model produces a refined transcript together with structured change information.

A deterministic validation layer is applied after the model response. It checks the proposed changes against the original segment and rejects unsafe modifications, including changes to numbers or negation, unsupported changes, and changes that are not properly represented. Thus, the refined transcript remains traceable to the raw STT output while correcting terminology and obvious recognition errors.

1.3.3 Meeting Documentation and Evidence-Grounded Extraction

The final stage converts the refined transcript into a structured meeting record.

Before documentation generation, a **Transformation layer** organizes the transcript into coherent topics and discussion units and identifies candidate decisions and action items with their supporting segment IDs. It also performs consensus analysis to distinguish confirmed agreement from proposals, unresolved points, and tentative statements. Contextual reconstruction is used where a decision depends on surrounding utterances rather than a single self-contained sentence.

The resulting structured guidance, together with the refined transcript, is passed to the second **DeepSeek deepseek-flash** stage. This stage has a distinct role from the refinement model: it generates the final meeting documentation, including a summary, organized minutes, decisions, and actionable tasks. Owners and deadlines are included only when they are explicitly supported by the meeting. Proposals, recommendations, conditional statements, and unresolved items are not promoted to confirmed decisions or assignments. ML Bootcamp

The final output is stored in both human-readable and machine-readable forms. Each decision and action can be traced back to the supporting transcript segments, allowing the user to inspect the evidence behind the generated record. This provides a traceable path from the original audio and raw transcript to the final meeting summary and action items.

1.3. Interactive Application:

- **Audio Upload:**
The application allows the user to upload an English-language meeting recording and start the complete processing pipeline from a single interface. ML Bootcamp
- **Pipeline Status:**
Processing progress is displayed stage-by-stage, covering transcription, transcript refinement, transformation, and documentation generation. The user can see whether each stage is processing, completed, or failed.
- **Transcript Review:**
The application provides access to both the **raw transcript** generated by the

STT system and the **refined transcript** produced after domain-aware correction. This allows the user to compare the two versions. ML Bootcamp

- **Meeting Documentation:**

The final interface presents the generated **summary, meeting minutes, decisions, and actionable tasks** in an organized format. ML Bootcamp

- **Evidence-Based Review:**

Decisions and action items can be traced back to their supporting transcript segments. The application displays the relevant speaker, timestamp, and transcript content, allowing the user to verify the extracted information.

- **Warnings and Uncertainty:**

Items with ambiguous or weaker supporting evidence are flagged for review instead of being presented as fully certain.

- **Downloadable Outputs:**

The application provides downloadable outputs for the different stages, including:

- Raw transcript
- Refined transcript
- Meeting summary
- Meeting minutes
- Decisions
- Actionable tasks
- Machine-readable structured meeting record

- **Human- and Machine-Readable Results:**

The final meeting record is available in both readable text formats and structured formats suitable for further processing. ML Bootcamp

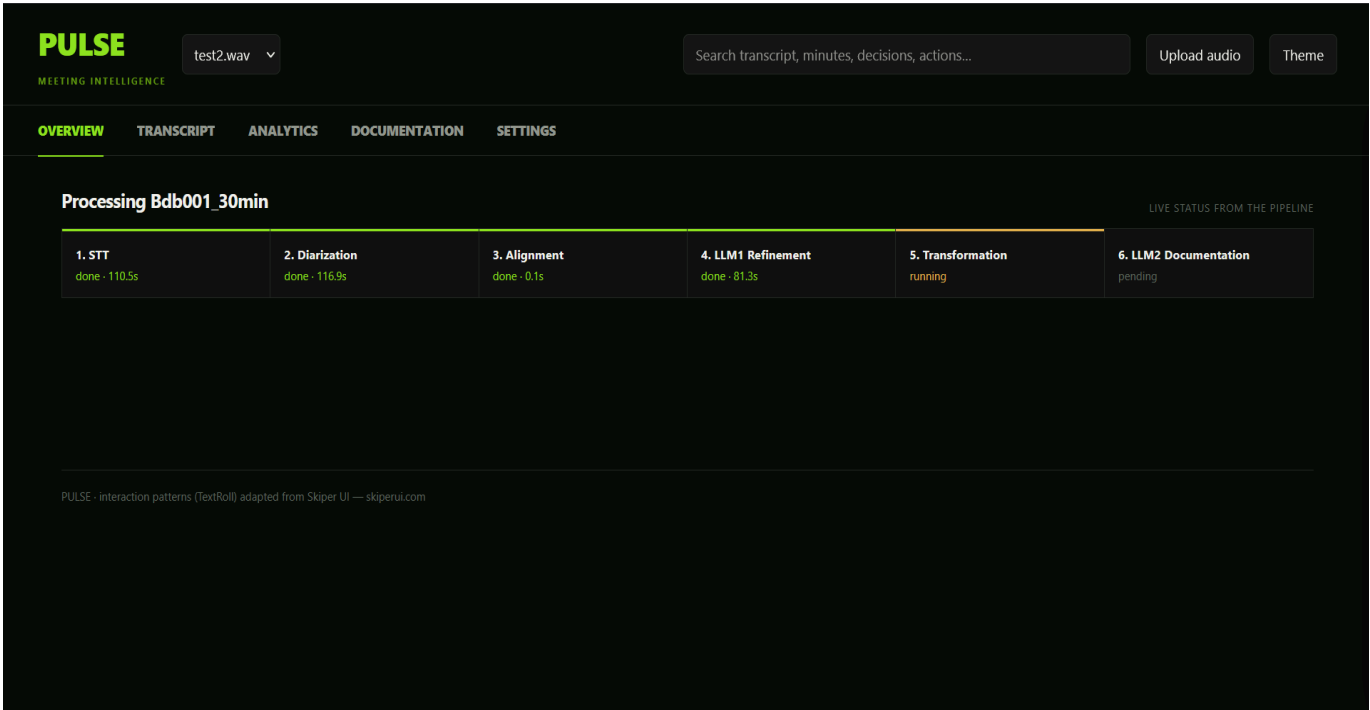
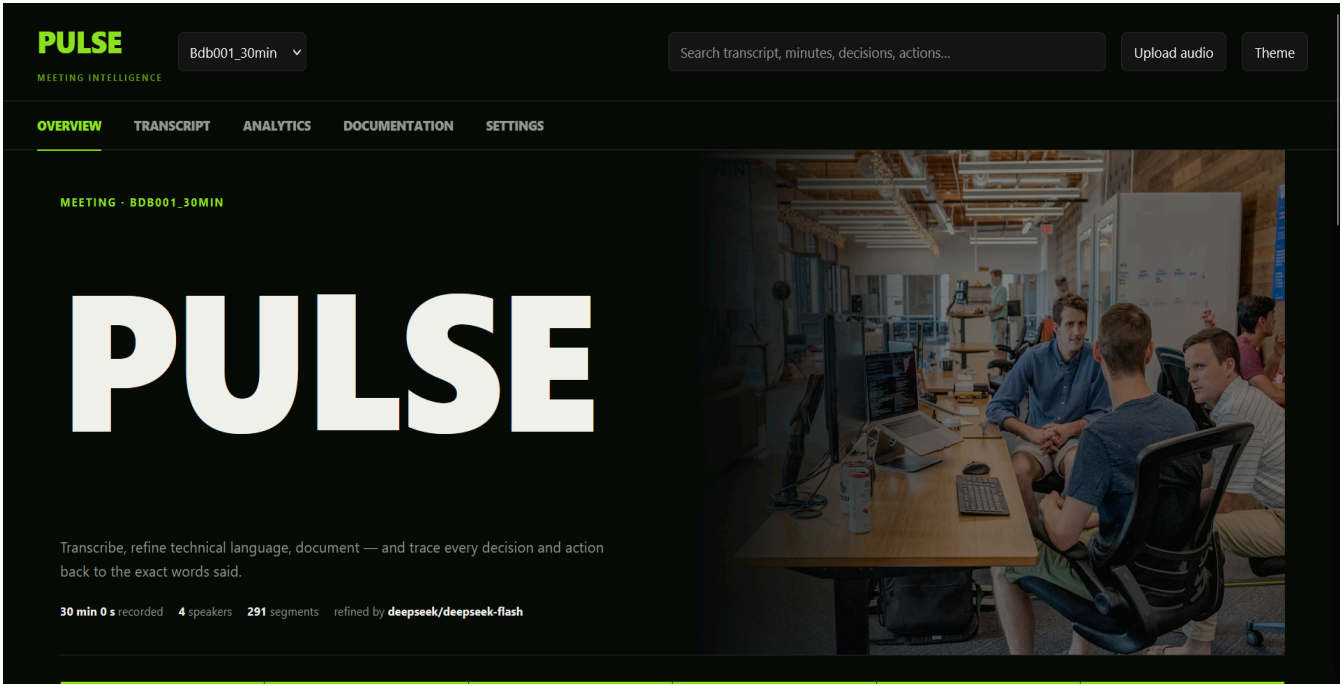
- **Error Handling:**

Unsupported, empty, unreadable, or unsuccessfully processed recordings are reported through clear error messages in the interface rather than exposing internal system errors. ML Bootcamp

- **Single End-to-End Workflow:**

The application connects all processing stages into one workflow, allowing a new recording to be uploaded and processed through to the final downloadable meeting record without manually running individual pipeline components. ML Bootcamp

1.5 Application Interface



PULSE

MEETING INTELLIGENCE

Bdb001_30min

▼

Search transcript, minutes, decisions, actions...

Upload audio

Theme

OVERVIEW

TRANSCRIPT

ANALYTICS

DOCUMENTATION

SETTINGS

Meeting timeline

0:00 → 29:59.5 · CLICK A BLOCK TO OPEN IT

Who spoke

Cited by documentation

Refined by LLM1

■ Speaker 00

■ Speaker 01

■ Speaker 02

■ Speaker 03

Refinement log

15 CHANGES · 6 REJECTED BY VALIDATION

SEG	ORIGINAL	REFINED	REASON (FROM LLM1)	CONF.
#87	Kepstrol	MFCC	Misrecognized technical term; context of speech features suggests MFCC (Mel-frequency cepstral coefficients).	0.9
#87	plp	PLP	Capitalization of acronym for perceptual linear prediction.	0.9
#97	and		Removed STT duplicate word	0.9

PULSE

MEETING INTELLIGENCE

test5

▼

Search transcript, minutes, decisions, actions...

Upload audio

Theme

OVERVIEW

TRANSCRIPT

ANALYTICS

DOCUMENTATION

SETTINGS

Documentation

FROM DOCUMENTATION_OUTPUT.JSON

SUMMARY

MINUTES

DECISIONS

ACTIONS

WARNINGS

Summary (.txt)

Minutes (.txt)

Decisions (.txt)

Action Items (.txt)

Final Record (.json)

MINUTE 1

Migration overview and event flow

The payment orchestration layer is being migrated from the monolith to a distributed architecture, with the constraint that duplicate charges must not be introduced while traffic is moved gradually. In the new design checkout publishes a PaymentRequested event to Kafka and the orchestration service consumes it, replacing the current synchronous call from checkout to the payment gateway adapter. Kafka partitions are keyed by payment_id, which gives ordering only for events with the same key, not global ordering across all payments. The consumer group will run three replicas initially, but only one consumer in the group owns a partition at a time.

segments 0, 1, 2, 3

MINUTE 2

Ledger, idempotency, and cache

The ledger remains on PostgreSQL, with serializable transactions used only around the balance mutation rather than the entire request lifecycle. The idempotency table is keyed by merchant_id and idempotency_key, with a unique constraint on merchant_id, idempotency_key, and operation_type; a retry with the same key but a different amount is rejected rather than silently updating the original request. Redis is only an optimization and a Redis hit must not be treated as proof that a payment exists in the ledger, since cache eviction or failover can remove the entry.

segments 4, 5, 6, 7

MINUTE 3

Payment history, history, and analytics

